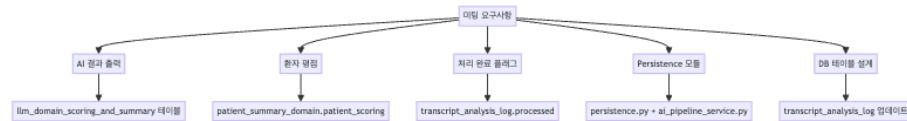


미팅 요구사항 → 구현 매핑

미팅 일자: 2026-04-16 | 구현 일자: 2026-04-16 ~ 2026-04-17
브랜치: feat/save-intermediate-results

1. 개요

이 문서는 2026-04-16 미팅의 각 요구사항을 실제 코드 변경사항에 매핑합니다.
각 요구사항에서 핵심 구현 포인트는 밑줄과 굵은 글씨로 강조하였습니다.



2. 요구사항별 구현 매핑

요구사항 1: “Guillermo 코드에서 AI result output 확인 필요하고 DB persis-tency”

원문: “check Guillermo’s code, ai result code output, and db per-sistency”

구현 요약: Guillermo의 ai_pipeline/ 모듈이 ai_pipeline_service.py를 통해 Docker 안에서 호출되며, 결과는 llm_domain_scoring_and_summary 테이블에 자동 저장됩니다.

상세 구현:

1. 호출 경로:

pipeline_runner.py (Step 9) → ai_pipeline_service.run_ai_scoring_and_summary()
→ ai_pipeline/pipeline.py → run_ai_pipeline()

2. Guillermo 코드가 생성하는 것 (도메인당):

- **ai_score** (0-5): GPT-4o가 “의사가 이 도메인의 위험을 얼마나 명확히 전달했는가”를 평가한 점수. 의사 페이지에서 상담 품질 지표로 표시됨 (GET /api/doctor/scores/average, /scores/trajectory).
- **score_explanation**: GPT-4o가 점수를 매긴 근거 (chain-of-thought reasoning). 내부 디버깅용.

- **extracted_estimate**: 의사가 말한 구체적 수치 추정값 (예: "24-25% risk of death", "13 years"). 의사 페이지에서 원본 추정값 리뷰용.
- **treatment**: 해당 추정값과 관련된 치료법 (예: "surgery", "radiation", 또는 NULL). 부작용 도메인(ed, inc, ius)에서만 값이 있음.
- **reformat_sentence**: GPT-4o가 환자가 이해할 수 있는 언어로 변환한 텍스트. 환자 페이지에서 AI 요약 카드로 표시됨 (GET /api/patient/ai-summary/{file}).

3. DB 저장 위치:

llm_domain_scoring_and_summary 테이블 — 환자당 5~9행 (도메인당 1~2행)

4. 검증 결과:

SID 10, 14, 15, 18, 33 — 총 33행 저장, 5명 환자, 5개 도메인 모두 처리 완료

llm_domain_scoring_and_summary			
SERIAL	id	PK	
INT	analysis_id	FK	FK → transcript_analysis_log
VARCHAR	patient_id		예: SID_10
VARCHAR	domain		cp, le, ed, inc, ius
INT	ai_score		0-5, GPT-4o 점수 → 의사 페이지
TEXT	score_explanation		GPT-4o의 ai_score 산출 근거
TEXT	extracted_estimate		예: 24-25% 또는 13년
VARCHAR	treatment		surgery, radiation, 또는 NULL
TEXT	source_sentence		원본 문장
TEXT	source_context		main 태그 포함 주변 문장
TEXT	reformat_sentence		환자 친화적 텍스트 → 환자 페이지
VARCHAR	source_filename		원본 파일
TIMESTAMPZ	created_at		자동 타임스탬프

요구사항 2: “환자 평점(rating) — 변경 필요한 부분”

원문: “there is couple of things we need to change, which is rating from patients”

원문: “things for the patient, the review part they are rating”

구현 요약: `patient_summary_domain.patient_scoring` 컬럼이 환자의 별점 평가를 저장합니다. 파이프라인에서는 NULL로 초기화되고, 환자가 재진 대시보드에서 직접 입력합니다.

상세 구현:

1. 저장 테이블: `patient_summary_domain`
2. 컬럼: `patient_scoring` (INT, 0-10)
 - 파이프라인 실행 시: NULL로 초기화 (아직 환자가 평가하지 않음)
 - 환자가 대시보드에서 별점 클릭 시: `PUT /api/patient/scoring` API로 업데이트
3. 프론트엔드:

`PatientFollowUpReportV31Re.tsx` → 각 도메인 (cp, le, ed, inc, ius) 별로 별점 (star rating) UI 표시
4. INSERT vs UPDATE:

UPDATE 방식 — “현재 평점”만 중요하므로 이전 값을 덮어씀. 평점 이력은 보존하지 않음.

patient_summary_domain			
VARCHAR	file		
PK_FK	VARCHAR		
speaker	PK_FK		
VARCHAR	domain	PK	예: cancer_prognosis
INT	display_order		1=cp 2=inc 3=ed 4=ius 5=le
TEXT	summary_text		도메인 요약
INT	patient_scoring		0-10 별점 — 환자가 입력
TEXT	patient_response		자유 텍스트 — 환자가 입력

중요 구분: `patient_scoring` ≠ `ai_score` - `patient_scoring` = 환자의 주관적 평가 (“의사가 이 주제를 잘 설명했나?”) → 환자 페이지에서 입력 - `ai_score` = GPT-4o의 객관적 문장 관련도 점수 (0-5) → 의사 페이지에서

표시 - 두 점수는 서로 다른 테이블, 다른 페이지, 다른 목적

요구사항 3: “처리 완료 여부 플래그 추가”

원문: “we also need to flag if it is been processed or not”

원문: “add if it is been processed or not”

구현 요약: `transcript_analysis_log` 테이블에 `processed` (BOOLEAN) + `processed_at` (TIMESTAMP) 컬럼을 추가했습니다.

상세 구현:

1. Step 8 (NLP 저장 시):
`persistence.py`의 `save_all()` → `processed=False`, `processed_at=NULL`로 저장
 2. Step 9 (AI 파이프라인 성공 시):
`ai_pipeline_service.py` → `processed=True`, `processed_at=datetime.now(UTC)`로 UPDATE
 3. Step 9 (AI 파이프라인 실패 시):
`processed=False` 유지 → 나중에 재처리 가능 (non-blocking)
 4. 추가 타이밍 컬럼:
`pipeline_started_at` = 파이프라인이 이 파일 처리를 시작한 시각
이를 통해 전체 소요 시간 계산: `processed_at - pipeline_started_at`
 5. 검증 결과:
`SID 10: processed=True, pipeline_started_at=05:26:14, processed_at=05:29:28 (3 14 )`
`SID 14: processed=True, pipeline_started_at=05:29:28, processed_at=05:32:40 (3 12 )`
`SID 15: processed=True, pipeline_started_at=05:32:40, processed_at=05:35:32 (2 52 )`
`SID 18: processed=True, pipeline_started_at=05:35:32, processed_at=05:38:36 (3 04 )`
`SID 33: processed=True, pipeline_started_at=05:38:36, processed_at=05:41:43 (3 07 )`
5명 환자 모두 `processed=True` 확인
-

요구사항 4: “main_complete_pipeline.py 실행하여 출력 이해”

원문: “run main_complete_pipeline.py and understand what is here (maybe the ai result variable in the Guillermo code)”

구현 요약: `SID 15` 파일로 `main_complete_pipeline.py`를 로컬에서 실행하여 Step 0~9 전체 출력을 확인했습니다. 각 Step의 DataFrame 행 수, 컬럼, 샘플 데이터를 디버그 로그로 출력하도록 코드를 수정했습니다.

상세 구현:

1. 실행 환경: conda prostate_cancer_py_3.10, NLP Docker 프록시 (localhost:9999), Azure OpenAI
2. 테스트 브랜치: test/complete-pipeline (AI_physician_patient_communication 레포, 커밋 582e046)

3. 코드 수정:

main_complete_pipeline.py에 각 Step 후 DataFrame 상세 로그 출력 추가 (행 수, 컬럼, 샘플)
 Path() 래핑 에러 수정 (line 83: config.output_path / patient_id
 → Path(config.output_path) / patient_id)

4. 실행 결과 (SID 15):

```
Step 0: 29 rows ( )
Step 1: 14 rows ( )
Step 2: 122 rows (R stringi )
Step 3: 122 rows (NLP 5 )
Step 4: 10 × 5 ( Top-K )
Step 5: 10 × 5 (context )
Step 6-9: 5 domains (AI pipeline - Scoring, Extraction, Filtering, Selection, Reformat)
```

5. 26개 출력 파일 생성:

```
data/output_test/SID_15/
  segmented_sentences.csv (Step 2: 122 )
  predictions_long.csv (Step 3: 122 × 5 )
  top10_by_outcome.xlsx (Step 4: 10 )
  top10_with_context.xlsx (Step 5: 10 + context)
  {domain}_extraction.csv (Step 7: )
  {domain}_filtering.csv (Step 8: )
  {domain}_result.csv (Step 9: + reformat)
  {domain}.xlsx ( )
```

요구사항 5: “무엇을 저장하고 어디로 가는지 확인”

원문: “we need the result and check what needs to be saved and where does it go”

원문: “you will see what is the thing that are going to patient and to doctor”

원문: “for doctor, i think we do not save anything”

구현 요약: 환자에게 가는 데이터와 의사에게 가는 데이터를 명확히 분리했습니다. 의사는 저장하지 않음 (읽기 전용).

환자에게 가는 것:

데이터	테이블	컬럼	API	설명
AI 위험 요약	llm_domain_scoring	patient_summary	POST /api/patient-summary/{file}	GPT-4o가 환자 진술을 언어로 변환한 텍스트. 예: “Your doctor noted that your risk of dying of prostate cancer is 24-25%.”
환자 평점	patient_summary	patient_score	GET /api/patient-scoring	환자가 재진 입력하는 0-10 별점
환자 응답	patient_summary	patient_responses	POST /api/patient-responses	환자가 직접 입력하는 자유 텍스트

의사에게 가는 것 (읽기 전용):

데이터	테이블	컬럼	API	설명
AI 품질 점수	llm_domain_scoring	doctor_score	GET /api/doctor-score	GPT-4o의 0-5 점수/average 의사 대시보드에 상당 품질 지표로 표시
상위 문장	doctor_sentence	sentence, class	GET /api/doctor-sentences/{file}/{speaker}	NLP Top 10 문장
점수 궤적	llm_domain_scoring	doctor_score	GET /api/doctor-score-trajectory	시간에 따른 상당 품질 변화 추이

의사는 저장하지 않음 — 의사 뷰는 읽기 전용(read-only). 의사가 문장을 rewrite하면 doctor_rewrite_log에 저장되지만 이것은 대시보드 UI에서 발생하며 파이프라인과 무관합니다.

요구사항 6: “Persistence 모듈 생성 — 키는 patient_id”

원문: “create module of persistency, and then call that module there and save those things to the db”

원문: “key should be patient id”

구현 요약: `persistence.py`의 `save_all()` 함수가 모든 DB 저장을 단일 트랜잭션으로 처리합니다. 키는 `patient_id`.

상세 구현:

1. 모듈: `persistence.py` — `save_all()` 함수 1개로 모든 DB 쓰기 담당
2. 호출: `pipeline_runner.py` Step 8에서 `persistence.save_all(Session, ...)` 한 줄로 호출
3. 키: `transcript_analysis_log.patient_id` (예: `SID_10`)
4. 트랜잭션: 단일 트랜잭션 — 모든 테이블에 성공하거나 모두 롤백
5. 저장하는 테이블 (한 번의 호출로):

순서	테이블	행 수	내용
1	<code>transcript_analysis_log</code>	1	분석 실행 기록 (<code>patient_id</code> , 설정값, <code>xlsx</code> , 타이밍, <code>processed</code> flag)
2	<code>sentence_prediction</code>	50	5 도메인 × 10 문장, NLP .pred_1 확률 + context
3	<code>doctor_sentence_view</code>	~47	중복 제거된 의사 문장 + 도메인 분류
4	<code>patient_summary</code>	1	환자 요약 기본 행
5	<code>patient_summary_domain</code>	5	도메인별 요약 + <code>patient_scoring(NULL)</code> + <code>patient_response(NULL)</code>

요구사항 7: “AI summary가 아니라 다른 이름 (AI reformat)”

원문: “make sure that you have the rating for the sentence, they call it now something different, it is not the ai summary (i think it

is ai reformat?)”

원문: “check the right language on the video”

구현 요약: “AI summary”라는 용어가 **reformat_sentence**로 변경되었습니다.
이것은 llm_domain_scoring_and_summary 테이블의 컬럼명입니다.

상세 구현:

1. 이전 용어: “AI summary”
2. 새 용어: **reformat_sentence**
3. 생성 과정:
ai_pipeline/reformat.py → GPT-4o가 Selection에서 선택된 1개 추정값을 환자가 이해할 수 있는 언어로 변환
4. 실제 예시:
 - 입력 (의사 원문): "the chances of you dying from this prostate cancer is coming out at about 24 percent"
 - GPT-4o Extraction: "24% risk of death without treatment"
 - GPT-4o Reformat: "Your doctor noted that your risk of dying of prostate cancer is 24-25%. With treatment, this risk decreases to 6%."
5. 프론트엔드에서의 사용:
PatientInitialVisitReportV35.tsx와 PatientFollowUpReportV31Re.tsx가 GET /api/patient/ai-summary/{file} API를 호출하면, routes_patient.py가 llm_domain_scoring_and_summary.reformat_sentence를 반환하고, 프론트엔드가 이것을 환자용 AI 요약 카드로 표시합니다.

요구사항 8: “테이블: id, sentence, domain, rating, processed flag”

원문: “Basically the table is the id, sentence, and then the domain, and then the rating, and then the flag to show processed or not. So basically we are doing the ground work. This is the saving step.”

구현 요약: 미팅에서 요청한 5개 필드가 두 테이블에 걸쳐 구현되었습니다.

미팅 요구사항과 실제 구현의 매핑:

미팅에서 요청한 것	실제 구현 위치	컬럼	설명
id	transcript_analysis_id	idg (SERIAL PK)	환자별 분석 실행 고유 ID
sentence	llm_domain_scoring_and_summary	sentence (TEXT)	AI 요약 평가한 원본 문장

미팅에서 요청한 것	실제 구현 위치	컬럼	설명
domain	llm_domain_scoring_and_summary	domain (VAR-CHAR)	cp, le, ed, inc, ius
rating	llm_domain_scoring_and_summary	rating (INT, 0-5)	GPT-4o의 문장 관련도 점수
processed flag	transcript_analysis_log	processed (BOOLEAN)	True = NLP + AI 완료

왜 두 테이블인가: - transcript_analysis_log = 환자 단위 (1 patient = 1 row) — 전체 파이프라인 메타데이터 + processed flag - llm_domain_scoring_and_summary = 도메인 단위 (1 patient × 5 domains = 5+ rows) — AI 결과 상세

이 두 테이블은 analysis_id (FK)로 연결됩니다:

transcript_analysis_log			
SERIAL	id	PK	환자 분석 실행
VARCHAR	patient_id		예: SID_10
TIMESTAMPZ	pipeline_started_at		처리 시작 시각
TIMESTAMPZ	analyzed_at		NLP 결과 저장 시각 (Step 8)
BOOLEAN	processed		NLP + AI 완료 시 True
TIMESTAMPZ	processed_at		AI 파이프라인 완료 시각
FLOAT	ai_overall_score		전체 도메인 ai_score 평균

도메인별 AI 결과

llm_domain_scoring_and_summary			
SERIAL	id	PK	도메인별 AI 결과
INT	analysis_id	FK	FK → transcript_analysis_log
VARCHAR	patient_id		예: SID_10
VARCHAR	domain		cp, le, ed, inc, ius
INT	ai_score		0-5 평점 (GPT-4o)
TEXT	source_sentence		평가된 문장
TEXT	reformat_sentence		환자 친화적 변환

3. 파이프라인 타이밍 (검증 완료)

SID 10: pipeline_started_at=05:26:14 → analyzed_at=05:26:30 → processed_at=05:29:28 (3 14)

SID 14: pipeline_started_at=05:29:28 → analyzed_at=05:29:39 → processed_at=05:32:40 (3 12)
 SID 15: pipeline_started_at=05:32:40 → analyzed_at=05:32:46 → processed_at=05:35:32 (2 52)
 SID 18: pipeline_started_at=05:35:32 → analyzed_at=05:35:41 → processed_at=05:38:36 (3 04)
 SID 33: pipeline_started_at=05:38:36 → analyzed_at=05:38:46 → processed_at=05:41:43 (3 07)

5명 환자 모두: processed=True

4. 변경된 파일

파일	변경 내용
pipeline_runner.py	transcript_service를 sentence_classification (R stringi)으로 교체. pipeline_started_at 기록, 중간 결과 파일 저장 (step0-step5) 추가.
persistence.py	초기 저장 시 pipeline_started_at + processed=False 추가.
ai_pipeline_service.py	AI 성공 시 processed=True + processed_at 설정. Azure 타임아웃 30분으로 증가.
models.py	TranscriptAnalysisLog에 pipeline_started_at, processed, processed_at 추가. 프론트엔드 사용 주석 추가.
database_schema.sql	transcript_analysis_log CREATE TABLE에 3개 새 컬럼 추가.
docker-compose.yml	sentence_classification/ 볼륨 마운트 추가. start_period 2400초로 확장.
Dockerfile	R + stringi 1.8.4 (번들 ICU 74.1) + rpy2 설치. prestart에 ICU 버전 검증 추가.
routes_transcript.py	transcript_service.analyze_transcript를 sentence_classification 기반으로 교체.
transcript_service.py	archive/로 이동 (더 이상 사용 안 함).

5. 잔여 작업

항목	상태	비고
처리 완료 플래그	완료	transcript_analysis_log.processed + processed_at

항목	상태	비고
환자 평점	존재	patient_summary_domain.patient_scoring (환자가 대시보드에서 입력)
AI reformat 의사 읽기 전용	완료 확인	llm_domain_scoring_and_summary.reformat_senten 의사 뷰는 저장 안 함 — DB에서 읽기만
통합 테스트	다음 주	End-to-end: 입력 → 파이프라인 → DB → 대시보드 표시
설문 변경	보류	미팅 결정에 따라 — 변경 확정 전까지 미착수